**Ex: 09** **DESIGN STUDENT CURD & PRODUCT INVENTORY** **Date: 02.03.2026**

**Aim:**

To develop a Student CRUD application using Express.js and to design a Product Inventory Dashboard using React JS integrated with Express REST API for performing Create, Read, Update, Search, and Delete operations.

**Description:**

The applications are developed using modern web technologies including **React.js** for frontend and **Express.js (Node.js)** for backend server development.

**Experiment 1: Student CRUD using Express**

A basic CRUD application is developed using Express.js without database integration. Student data is managed in server memory and CRUD operations are performed using REST API routes.

**Concepts Used:**

• Express.js – To create server and API routes
• REST API – GET, POST, PUT, DELETE methods
• Middleware – express.json() for parsing request data
• Routing – Handling different endpoints for CRUD operations
• In-memory Data Storage – Managing student records without database

**Code:**
Server.js:

```
const express = require("express");
const cors = require("cors");
const app = express();
app.use(cors());
app.use(express.json());

let products = [
  { id: 1, name: "Laptop", price: 50000, quantity: 10 },
  { id: 2, name: "Mobile", price: 20000, quantity: 25 },
  { id: 3, name: "Headphone", price: 2000, quantity: 50 }
];
app.get("/products", (req, res) => {
  res.json(products);
});
app.get("/products/search", (req, res) => {
  const keyword = req.query.name;
  if (!keyword) return res.json(products);

  const filtered = products.filter(p =>
    p.name.toLowerCase().includes(keyword.toLowerCase())
  );
  res.json(filtered);
});
app.delete("/products/:id", (req, res) => {
    const id = parseInt(req.params.id);
```

```
  const index = products.findIndex(p => p.id === id);

  if (index !== -1) {
    products.splice(index, 1);
    res.json({ message: "Product Deleted Successfully" });
  } else {
    res.status(404).json({ message: "Product Not Found" });
  }
});

app.put("/products/:id", (req, res) => {
  const id = parseInt(req.params.id);
  const { name, price, quantity } = req.body;

  const product = products.find(p => p.id === id);

  if (product) {
    product.name = name;
    product.price = Number(price);
    product.quantity = Number(quantity);
    res.json({ message: "Product Updated Successfully" });
  } else {
    res.status(404).json({ message: "Product Not Found" });
  }
});

app.post("/products", (req, res) => {
  const { name, price, quantity } = req.body;

  const newProduct = {
    id: products.length + 1,
    name,
    price: Number(price),
    quantity: Number(quantity)
  };

  products.push(newProduct);
  res.json({ message: "Product Added Successfully" });
});
app.listen(5000, () => {
  console.log("Server running on http://localhost:5000");
});
```

**Experiment 2: Product Inventory Dashboard using React & Express**

A single page application (SPA) is developed using React.js for managing product inventory. The frontend communicates with the Express backend through REST API.

**React & Backend Concepts Used:**

• React Functional Components – To create reusable UI components (Dashboard, AddProduct, DeleteProduct)
• useState Hook – To manage product data, search input, and edit state

• useEffect Hook – To fetch data when component loads
• Event Handling – To handle form submission and button clicks
• Conditional Rendering – To display edit form dynamically
• REST API Integration – Fetching and updating data using HTTP methods
• CRUD Operations – Create, Read, Update, Delete products
• Search Functionality – Filtering products using query parameters
• CSS Styling – Minimal and neat styling for clean UI layout

**Code:**

**AddProduct.jsx:**

```
import React, { useState } from "react";
export default function AddProduct({ refreshProducts }) {
  const [product, setProduct] = useState({
    name: "",
    price: "",
    quantity: ""
  });
  const handleChange = (e) => {
    setProduct({ ...product, [e.target.name]: e.target.value });
  };
  const handleSubmit = async (e) => {
    e.preventDefault();
    await fetch("http://localhost:5000/products", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(product)
    });
    setProduct({ name: "", price: "", quantity: "" });

    if (refreshProducts) refreshProducts();
  };
  return (
    <div className="edit-box">
      <h3>Add Product</h3>
      <form onSubmit={handleSubmit}>
        <input
          type="text"
          name="name"
          placeholder="Product Name"
          value={product.name}
          onChange={handleChange}
          required
        />
        <input
          type="number"
```

```
        name="price"
        placeholder="Price"
        value={product.price}
        onChange={handleChange}
        required
       />
       <input
        type="number"
        name="quantity"
        placeholder="Quantity"
        value={product.quantity}
        onChange={handleChange}
        required
       />
       <button type="submit">Add</button>
     </form>
   </div>
  );
}
```

**DeleteProduct.jsx:**

```
import React, { useState } from "react";
export default function DeleteProduct({ refreshProducts }) {
  const [id, setId] = useState("");
  const handleDelete = async (e) => {
    e.preventDefault();
    await fetch(`http://localhost:5000/products/${id}`, {
      method: "DELETE"
    });
    setId("");
    if (refreshProducts) refreshProducts();
  };
  return (
    <div className="edit-box">
      <h3>Delete Product</h3>
      <form onSubmit={handleDelete}>
        <input
          type="number"
          placeholder="Enter Product ID"
          value={id}
          onChange={(e) => setId(e.target.value)}
          required
        />
        <button type="submit">Delete</button>
      </form>
    </div>
  );
```

```
}
```

**Dashboard.jsx:**

```
import React, { useEffect, useState } from "react";
import "./App.css";
import AddProduct from "./AddProduct";
import DeleteProduct from "./DeleteProduct";

export default function Dashboard() {
  const [products, setProducts] = useState([]);
  const [search, setSearch] = useState("");
  const [editProduct, setEditProduct] = useState(null);

  const fetchProducts = async () => {
    const res = await fetch("http://localhost:5000/products");
    const data = await res.json();
    setProducts(data);
  };
  const searchProduct = async () => {
    if (!search.trim()) {
      fetchProducts();
      return;
    }
    const res = await fetch(
      `http://localhost:5000/products/search?name=${search}`
    );
    const data = await res.json();
    setProducts(data);
  };
  const updateProduct = async () => {
    await fetch(`http://localhost:5000/products/${editProduct.id}`, {
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(editProduct),
    });
    setEditProduct(null);
    fetchProducts();
  };
  useEffect(() => {
    fetchProducts();
  }, []);
  return (
    <div className="container">
      <h2>Product Inventory Dashboard</h2>

      <AddProduct refreshProducts={fetchProducts} />
      <DeleteProduct refreshProducts={fetchProducts} />
```

```
<input
  type="text"
  placeholder="Search Product"
  value={search}
  onChange={(e) => setSearch(e.target.value)}
/>
<button onClick={searchProduct}>Search</button>
<button onClick={fetchProducts}>Reset</button>
<table>
  <thead>
    <tr>
      <th>ID</th>
      <th>Name</th>
      <th>Price</th>
      <th>Quantity</th>
      <th>Edit</th>
    </tr>
  </thead>
  <tbody>
    {products.map((p) => (
      <tr key={p.id}>
        <td>{p.id}</td>
        <td>{p.name}</td>
        <td>{p.price}</td>
        <td>{p.quantity}</td>
        <td>
          <button onClick={() => setEditProduct({ ...p })}>
            Edit
          </button>
        </td>
      </tr>
    ))}
  </tbody>
</table>
{editProduct && (
  <div className="edit-box">
    <h3>Edit Product</h3>
    <input
      value={editProduct.name}
      onChange={(e) =>
        setEditProduct({ ...editProduct, name: e.target.value })
      }
    />
    <input
      type="number"
```

```
          value={editProduct.price}
          onChange={(e) =>
            setEditProduct({ ...editProduct, price: e.target.value })
          }
        />
        <input
          type="number"
          value={editProduct.quantity}
          onChange={(e) =>
            setEditProduct({ ...editProduct, quantity: e.target.value })
          }
        />
        <button onClick={updateProduct}>Update</button>
      </div>
    )}
  </div>
 );
}
```

**App.jsx:**

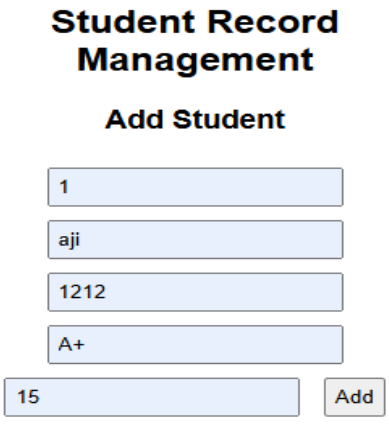
```
import React, { useEffect, useState } from "react";
import "./App.css";
import Dashboard from "./Dashboard";
function App() {
 return <>
  <Dashboard/>
 </>
}
export default App;
```

App.css:

```
body {
 font-family: Arial, sans-serif;
 background: #f4f6f8;
 margin: 0;
 display: flex;
 justify-content: center;   /* horizontal center */
 align-items: center;       /* vertical center */
 min-height: 100vh;
}
.container {
 width: 80%;
 max-width: 800px;
 text-align: center;
}
input {
 padding: 6px;
 margin: 5px;
}
```

```
button {
  padding: 6px 12px;
  margin: 5px;
  border: none;
  background: #007bff;
  color: white;
  cursor: pointer;
}
button:hover {
  background: #0056b3;
}
table {
  width: 100%;
  border-collapse: collapse;
  background: white;
  margin-top: 15px;
}
th, td {
  padding: 8px;
  border: 1px solid #ddd;
}
th {
  background: #007bff;
  color: white;
}
.edit-box {
  margin-top: 20px;
  padding: 15px;
  background: white;
  display: inline-block;
}
```

**Output:**




Figure(a) Adding Student Data

## Student List

| ID | Name | Roll No | Grade | Age |
|---|---|---|---|---|
| 1 | aji | 1212 | A+ | 15 |
| 2 | Riya | 1223 | A+ | 14 |
| 2 | Vikram | 1245 | A | 15 |

Go Back

Figure(b)**:** Selection of Records

## Student List

| ID | Name | Roll No | Grade | Age |
|---|---|---|---|---|
| 1 | aji | 1212 | S | 15 |
| 2 | Riya | 1223 | A+ | 14 |
| 2 | Vikram | 1245 | A | 15 |

Go Back

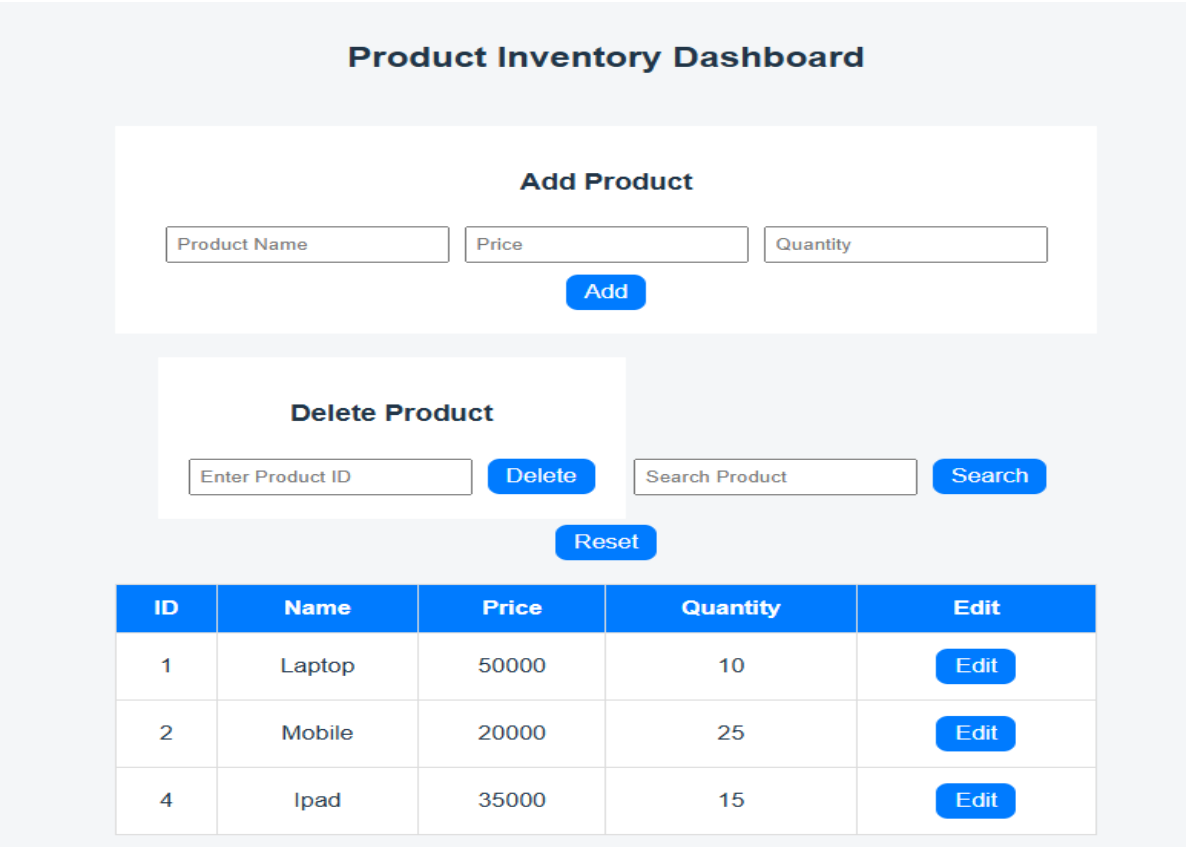
Figure(c)**:** Updation on Records

## Student List

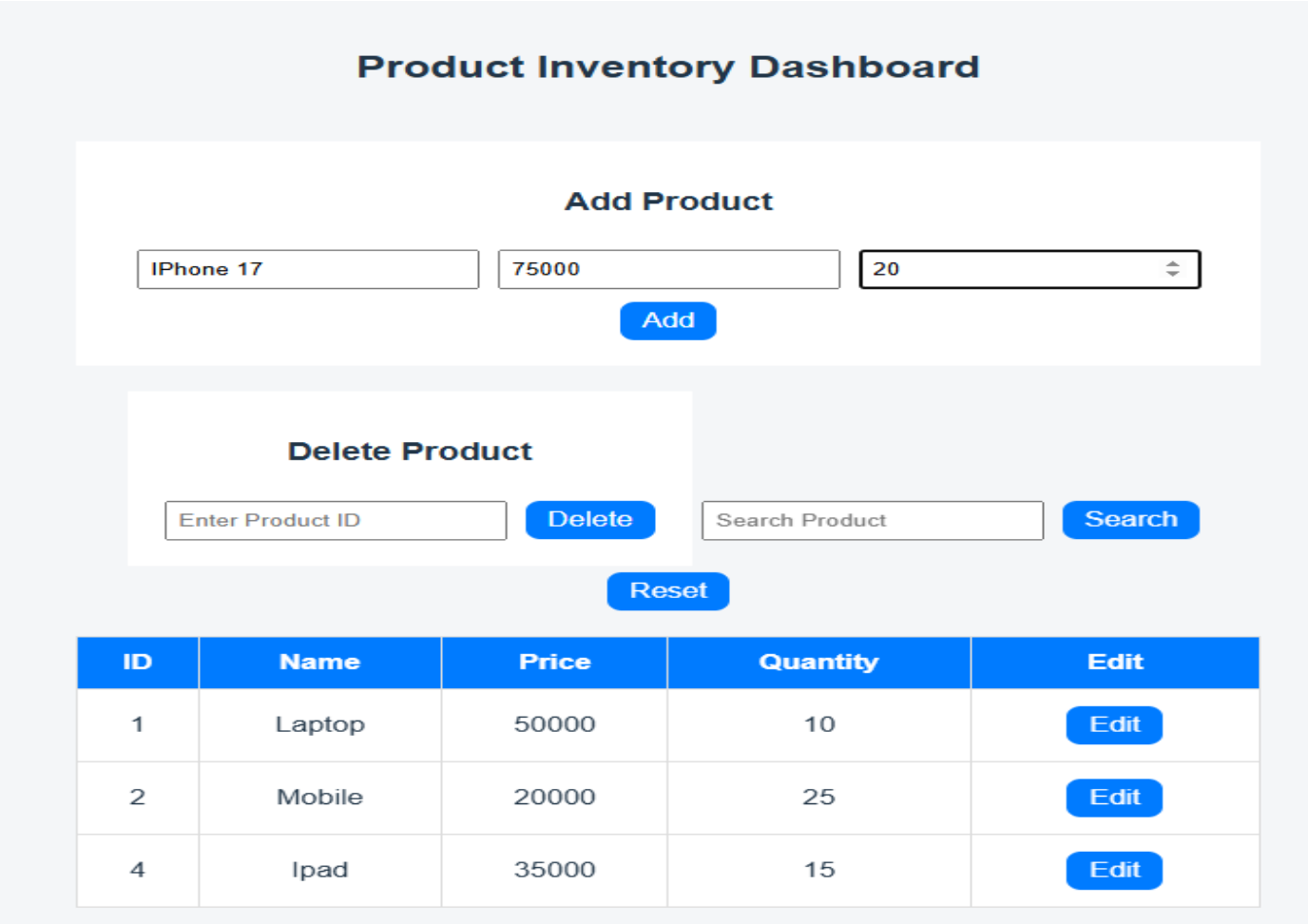
| ID | Name | Roll No | Grade | Age |
|---|---|---|---|---|
| 1 | aji | 1212 | S | 15 |
| 2 | Riya | 1213 | A+ | 14 |

Go Back

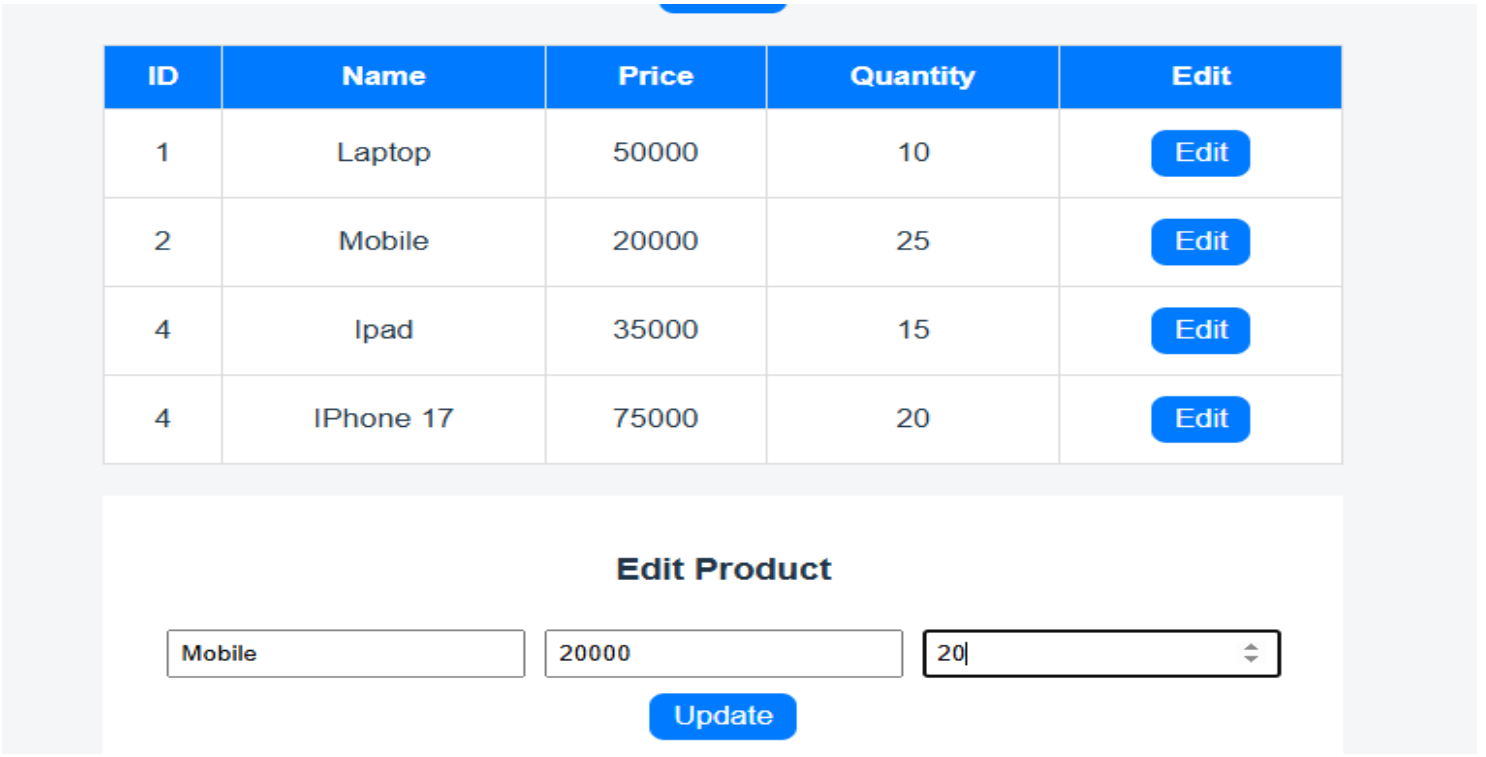
Figure(d)**:** Deletion on Records



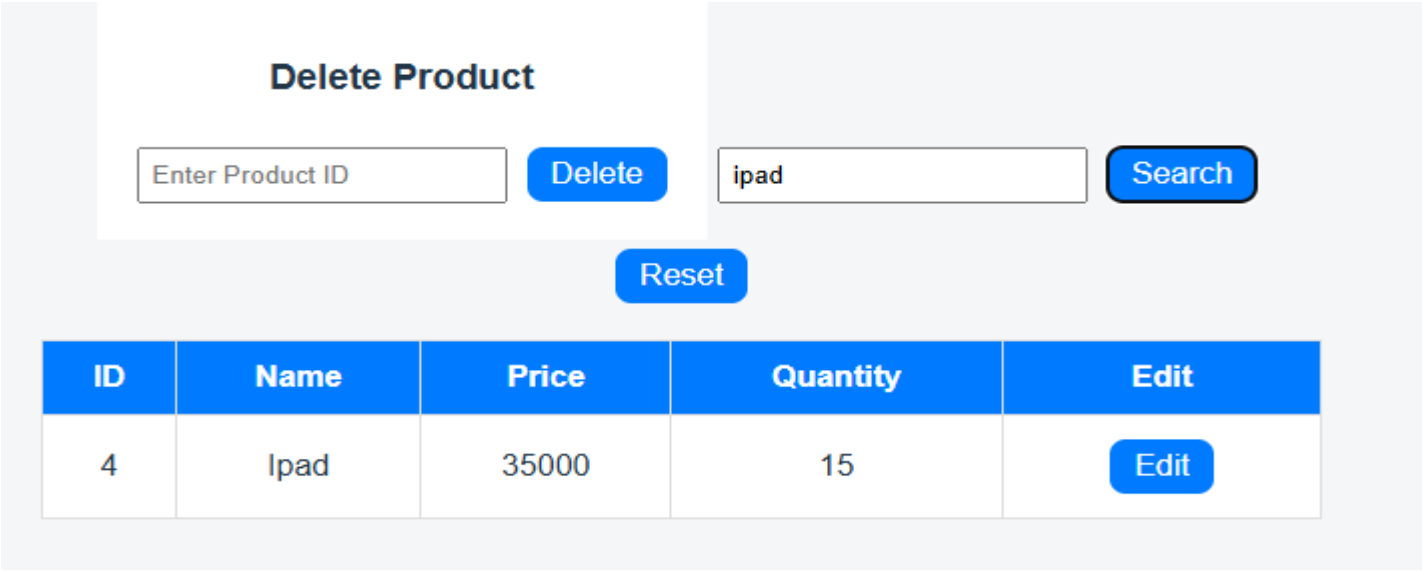

Figure(e): Dashboard of Product Inventory



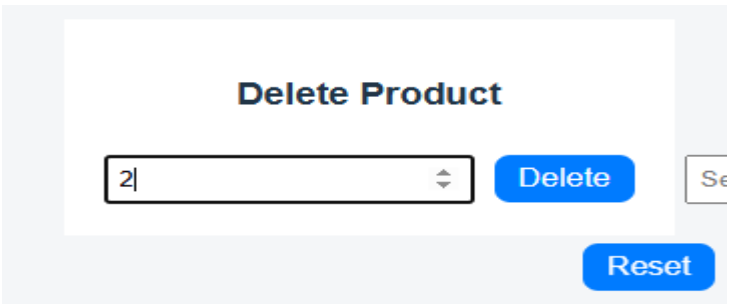
Figure(f): Adding Product in Inventory



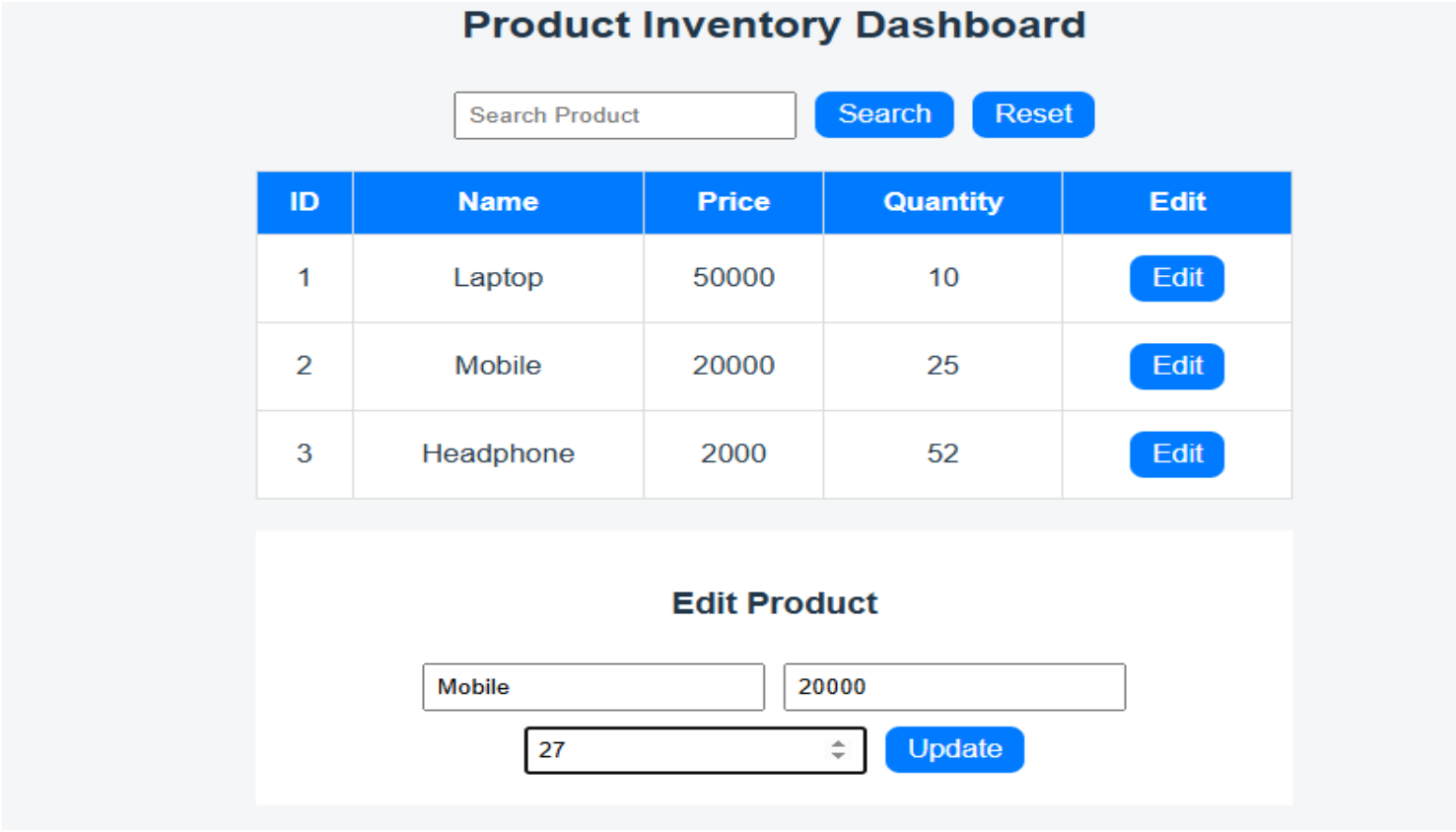

Figure(g): Update Product in Inventory




Figure(h): Search on Product Inventory




Figure(i): Delete Product in Inventory




Figure(j)**:** Update on Product Inventory

**Result:**

Thus, the Student CRUD application using Express and the Product Inventory Dashboard using React with Express REST API were successfully developed and executed, demonstrating full CRUD functionality and RESTful communication between frontend and backend.